

Implementacija stohastičkih SAT rešavača

10. mart 2026.

Sažetak

SAT problem je jedan od najvažnijih problema u oblasti logike. SAT rešavači, odnosno procedure za rešavanje SAT problema, kao izlaz vraćaju informaciju da li je formula zadovoljiva, a neki od njih i zadovoljavajuću valuaciju. Potpuni rešavači garantuju rezultat, ali zbog sistematskog pretraživanja rešenja nisu efikasni na određenim instancama. Za potrebe efikasnog rešavanja takvih instanci razvijeni su stohastički SAT rešavači, koji koriste probabilističke metode u svojim odlukama i time ubrzavaju pregled prostora mogućih rešenja. U ovom radu biće predstavljene osnove ideje stohastičkog pristupa i implementacija WalkSAT stohastičkog rešavača.

1 Uvod

SAT problem predstavlja problem ispitivanja zadovoljivosti formule, odnosno pronalaženje istinitosnih vrednosti promenljivih koje se javljaju u toj formuli, a za koje je data formula tačna. Procedure koje se koriste za njegovo rešavanje nazivaju se *SAT rešavači*.

Po načinu rešavanja problema, SAT rešavači se mogu podeliti na potpune i nepotpune - stohastičke. Za datu iskaznu formulu, deterministički (potpuni) SAT rešavači sistematski pretražuju prostor mogućih vrednosti koje se mogu dodeliti promenljivama. Najčešće korišćen algoritam na kome su zasnovani potpuni SAT rešavači je CDCL algoritam[1]. Ovaj algoritam karakteriše postepena izgradnja valuacije, sposobnost vraćanja (*backtracking*) na prethodna stanja (stanje je jedna dodela vrednosti promenljivama) i sposobnost učenja na osnovu otkrivenih konflikata tokom pretrage. Zbog toga deterministički rešavač garantuje da će nakon konačnog broja koraka vratiti rezultat. Kao procedura odlučivanja vratiće informaciju da je formula zadovoljiva (SAT) ili nezadovoljiva (UNSAT), a kao procedura pretrage, za zadovoljivu instancu problema, vratiće i njen model.

S obzirom da rešavači pretražuju potencijalno ogromno stablo pretrage, u praksi se pokazalo da su neefikasni na nekim tipovima velikih i "teških" SAT instanci. Uočeno je da je nekada lakše pronaći model isprobavanjem potpunih valuacija nego njihovom sistematskom izgradnjom. Tako nastaju stohastički (nepotpuni) SAT rešavači. Stohastički rešavači ne koriste sistematsku pretragu, već u svakom koraku nasumično generišu narednu potpunu valuaciju i proveravaju da li je ona model formule. Tipično su zasnovani na lokalnoj pretrazi (*Local Search*): kreću od nekog nasumičnog rešenja i u svakom koraku prave malu izmenu (*flip*) tako što menjaju vrednost nekoj od promenljivih. Ključni deo algoritma je heuristika za izbor promenljive za *flip*, na osnovu koje nastaju različiti sto-

hastički SAT rešavači. Iako efikasno pronalaze rešenje, nisu u mogućnosti da pokažu da rešenje ne postoji.

U narednom poglavlju uvešćemo osnovne termine i pojmove neophodne za razumevanje problema i rešenja. Zatim ćemo u poglavlju 3 opisati algoritam i objasniti ulogu svakog njegovog koraka. U poglavlju 4 ponuđena je jedna moguća implementacija u jeziku C++. Rad se završava poglavljem 5 u kojem ćemo sumirati rezultate istraživanja.

2 Osnove

Da bismo razmatrali problem zadovoljivosti, potrebno je da se upoznamo sa osnovnim pojmovima i teorijskom osnovom iskazne logike.

Iskazna logika proučava *iskaze* i odnose između njihovih istinitosnih vrednosti. *Iskazi* su tvrdjenja koja mogu biti tačna ili netačna. Elementarni iskazi su *iskazna slova* od kojih se, primenom *logičkih veznika* grade *složeni iskazi* i od čije istinitosne vrednosti zavisi istinitosna vrednost složenog iskaza u kojem figurišu.[4]

Valuaciju definišemo kao funkciju koja skup svih atoma preslikava u dvočlan skup $\{0,1\}$ odnosno svakom atomu dodeljuje vrednost tačno ili netačno. Formula je *zadovoljiva* ako postoji bar jedna valuacija u kojoj je ona tačna. Takva valuacija naziva se model formule. Formula je *nezadovoljiva* ako nema nijedan model.[4]

Pronalaženje modela formule naziva se *SAT problem - Satisfiability problem*, a implementacije procedura *SAT rešavači*.

SAT rešavači zahtevaju da formula koja im se prosleđuje bude u *konjunktivnoj normalnoj formi*. Formula je u *konjunktivnoj normalnoj formi* ako je oblika:

$$C_1 \wedge C_2 \wedge \dots \wedge C_n$$

pri čemu su $C_1 \dots C_n$ *klauze*, odnosno oblika su

$$L_{i1} \vee L_{i2} \vee \dots \vee L_{im}$$

pri čemu su L_{ij} *literal* - atomi ili negacije atoma.

3 Opis metode

Stohastički SAT rešavači zasnovani su na lokalnoj pretrazi (*Local Search*). Lokalna pretraga je optimizaciona tehnika koja u svakom koraku pokušava da poboljša vrednost ciljne funkcije. Uslov zaustavljanja algoritma je pronalazak modela ili ispunjenost uslova - maksimalan broj pokušaja, maksimalan broj zamena u jednom pokušaju itd.[2] Algoritam 1 prikazuje pseudokod lokalne pretrage u opštem obliku.

Algorithm 1 Local Search SAT

```
1: procedure LOCAL_SEARCH(SAT_instance, condition)
2:   valuation  $\leftarrow$  generate_random_valuation(SAT_instance)
3:   while condition do
4:     result  $\leftarrow$  check_satisfiability(SAT_instance, valuation)
5:     if result then
6:       return valuation
7:     end if
8:     variable_to_flip  $\leftarrow$  choose_variable_to_flip(SAT_instance, valuation)
9:     valuation[variable_to_flip]  $\leftarrow$  !valuation[variable_to_flip]
10:  end while
11:  return Unknown
12: end procedure
```

U okviru SAT problema, ciljna funkcija nam je razlika u broju trenutno nezadovoljenih klauza i broja klauza koje će biti nezadovoljive ukoliko flipujemo izabranu promenljivu. Pored ciljne funkcije, pratimo i vrednosti *breakcount* - broj klauza koje su zadovoljene u trenutnoj valuaciji, ali će postati nezadovoljene ako flipujemo izabranu promenljivu i *makecount* - broj klauza koje nisu zadovoljene u trenutnoj valuaciji, ali će postati zadovoljene ako flipujemo izabranu promenljivu.[3]

Izbor naredne promenljive zavisi od heuristike koja se koristi i to je ono u čemu nam se rešavači zapravo razlikuju, na primer:

1. GSAT - razmatra sve promenljive iz svih nezadovoljenih klauza i uzima onu koja minimizuje broj nezadovoljenih klauza, odnosno ima maksimalni *diffscore*
2. WalkSAT - uzima jednu nezadovoljenu klauzu i bira iz nje jednu promenljivu koju flipuje - ako postoje promenljive kojima je *breakcount* 0, nasumično bira jednu od njih; inače sa verovatnoćom p bira jednu promenljivu iz klauze nasumično, odnosno sa verovatnoćom $1-p$ bira promenljivu koja ima najmanji *breakcount* u klauzi.

Da bismo razumeli kako algoritmi rade, pokazaćemo postupak koraka izbora promenljive kroz primer. Posmatramo formulu:

$$(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

i valuaciju:

$$x_1 : 1, x_2 : 0, x_3 : 0$$

Broj nezadovoljenih klauza u ovoj valuaciji je 1. GSAT bi razmatrao zamenu vrednosti promenljivih koje figurišu u svim nezadovoljenim klauzama, ali ovde imamo samo jednu. Za svaku od promenljivih izračunamo *diffscore* i dobijamo sledeće vrednosti :

$$x_1 : 1, x_2 : 0, x_3 : 1$$

stoga zaključujemo da flipujemo promenljive x_1 ili x_3 .

Sa druge strane, WalkSAT prvo nasumično bira jednu od nezadovoljenih klauza, ali ovde imamo samo jednu, klauzu 4. Potencijalne promenljive za flipovanje su x_1 , x_2 i x_3 . Promenljive x_1 i x_3 imaju *breakcount* 0, a x_2 *breakcount* 1, pa nasumično biramo x_1 ili x_3 i *flip*-ujemo joj vrednost u valuaciji.

4 Implementacija

U ovom poglavlju predstavimo implementaciju prethodno opisanog WalkSAT rešavača, u programskom jeziku C++. Implementacija je organizovana u dva fajla, glavni cpp fajl u kome se nalazi implementacija algoritma i main funkcija koja pokreće rešavač nad primerom i pomoćni hpp fajl koji sadrži implementacije funkcija korišćenih u samom algoritmu.

Kako SAT rešavači koriste DIMACS formu, literale ćemo apstrahovati kao int vrednosti, kao i same promenljive, ali treba voditi računa da je promenljiva uvek jednaka apsolutnoj vrednosti literala, a ne nužno samom literalu. Klauze će biti nizovi literala, a normalna forma formule niz klauza. Valuaciju predstavljamo mapom koja kao ključeve uzima promenljive, a vrednosti mogu biti true ili false.

Funkcija **WalkSAT** predstavlja centar implementacije. Kao argumente prihvata formulu u KNF obliku, granicu zaustavljanja algoritma i verovatnoću p koja nam određuje način izbora promenljive.

Da bismo obezbedili nasumičnost u algoritmu, implementirani su generatori kroz interfejs koji pruža C++. Jedan generator koji koristimo u samom algoritmu generiše vrednosti između 0 i 1, a zatim tu vrednost koristimo u algoritmu da bismo izabrali jedan od dva načina izbora promenljive iz izabrane nezadovoljene klauze: nasumičan izbor promenljive ili izbor promenljive sa najmanjim *breakcount*-om. Pored toga, potrebni su nam i generatori u svakoj funkciji gde iz niza promenljivih/klauza nasumično biramo jedan element.

Listing 1: WalkSAT

```
1 #include "WalkSAT.h"
2 #include <optional>
3 #include <iostream>
4
5
6 uniform_real_distribution<> dis(0.0, 1.0);
7
8 optional<Valuation> WalkSAT(NormalForm& SAT_instance, int condition, double probability){
9
10     Valuation valuation = generate_random_valuation(SAT_instance);
11     Variable var_to_flip;
12
13     while(condition > 0){
14
15         if(is_satisfiable(SAT_instance, valuation)){
16             return valuation;
17         }
18
19         vector<Clause> unsatisfied_clauses = find_unsatisfied_clauses(SAT_instance, valuation);
20
21         Clause unsat_clause = choose_random_unsatisfied_clause(unsatisfied_clauses);
22
23         map<Variable, int> breakcounts;
24         for(auto literal : unsat_clause){
25             int var = abs(literal);
26             breakcounts[var] = calculate_breakcount(SAT_instance, var, valuation);
27         }
28
29         if(zero_breakcount_variables(breakcounts)){
30             var_to_flip = choose_random_zero_breakcount_variable_to_flip(breakcounts);
31         } else{
32             double p = dis(gen);
33
34             if(p < probability){
35                 var_to_flip = choose_random_variable_to_flip(unsat_clause);
36             } else{
37                 auto it = min_element(breakcounts.begin(), breakcounts.end(), [](const auto& p1, const auto&
38                                     p2) { return p1.second < p2.second;});
39                 var_to_flip = it->first;
40             }
41
42             valuation[var_to_flip] = !valuation[var_to_flip];
43             condition--;
44         }
45
46         return nullopt;
47     }
48
49     int main(){
50
51         double probability = 0.4;
```

```

52 NormalForm formula = {
53     {1, 2, 3}, {-1, -2, 4}, {1, -3, 5}, {-2, 3, -4}, {2, 4, -5},
54     {-1, 3, 5}, {1, -4, -5}, {-3, 4, 5}, {2, -3, -5}, {-1, -2, 3},
55     {1, 4, -5}, {-2, -4, 5}, {3, -4, -5}, {-1, 3, -4}, {2, 3, 4},
56     {1, 5, -2}, {-3, -5, 2}, {4, -1, -3}, {-5, 1, 4}, {2, -4, 6}
57 };
58
59
60 if(WalkSAT(formula, 10, probability)){
61     cout << "SAT" << endl;
62 } else{
63     cout << "Unknown" << endl;
64 }
65
66 return 0;
67 }

```

Algoritam počinje generisanjem nasumične valuacije, nasumičnim izborom vrednosti true ili false za svaku promenljivu(funkcija *generate_random_valuation*). Zatim se koraci u petlji ponavljaju sve dok nije ispunjen uslov zaustavljanja ili dok ne pronađemo model. U ovom primeru implementacije uslov zaustavljanja će nam biti maksimalan broj zamena vrednosti promenljivih. Petlja se sastoji od:

1. provere da li trenutna valuacija zadovoljava formulu(funkcija *is_satisfiable*) i ukoliko ne zadovoljava:
2. izbora promenljive za flip

U okviru hpp fajla, implementirane su sledeće pomoćne funkcije iz glavne petlje algoritma:

1. *find_unsatisfied_clause* - pronalaženje nezadovoljenih klauza
2. *choose_random_unsatisfied_clause* - nasumičan izbor jedne od nezadovoljenih klauza
3. *calculate_breakcount* - računanje *breakcount* vrednosti
4. *zero_breakcount_variables* - provera da li medju promenljivama iz nezadovoljene klauze ima onih sa *breakcount* vrednošću 0
5. *choose_random_zero_breakcount_variable_to_flip* - nasumičan izbor jedne od promenljivih kojoj je *breakcount* vrednost 0
6. *choose_random_variable_to_flip* - nasumičan izbor jedne od promenljivih iz izabrane nezadovoljene klauze
7. funkcija za izbor promenljive sa najmanjom *breakcount* vrednošću, iz nezadovoljene klauze - implementirano kroz ugrađenu funkciju *min* u C++ jeziku, pri čemu se vrši iteracija kroz mapu {promenljiva : breakcount} koja je povratna vrednost funkcije *calculate_breakcount*

Pri generisanju prve nasumične valuacije, poziva se funkcija *getVariables* koja prolazi kroz sve klauze formule i vraća skup svih promenljivih koje se u njoj javljaju.

U nastavku su prikazane implementacije struktura podataka i pomoćnih funkcija u hpp fajlu.

Listing 2: WalkSAT

```

1  #ifndef WALKSAT_H
2  #define WALKSAT_H
3
4  #include <vector>
5  #include <string>
6  #include <map>
7  #include <set>
8  #include <random>
9
10 using namespace std;
11
12 using Variable = int;
13 using Literal = int;
14
15 using Clause = vector<Literal>;
16 using NormalForm = vector<Clause>;
17
18 using Valuation = map<Variable, bool>;
19
20 random_device rd;
21 mt19937 gen(rd());
22
23 set<Variable> getVariables(NormalForm& SAT_instance){
24
25     set<Variable> variables;
26
27     for(const auto& clause : SAT_instance){
28         for(auto literal : clause){
29             if(variables.find(abs(literal)) == variables.end()){
30                 variables.insert(abs(literal));
31             }
32         }
33     }
34
35     return variables;
36 }
37
38
39 Valuation generate_random_valuation(NormalForm& SAT_instance){
40
41     set<Variable> variables = getVariables(SAT_instance);
42     uniform_int_distribution<> dis(0, 1);
43
44     Valuation valuation;
45
46     for(const auto& variable : variables){
47         int p = dis(gen);
48         valuation[variable] = p;
49     }
50
51     return valuation;
52 }
53
54 bool is_satisfiable(NormalForm& SAT_instance, Valuation& valuation){
55
56     bool formula = true;
57     for(const auto& clause : SAT_instance){
58
59         bool clause_value = false;
60         for(auto literal : clause){
61             if(valuation[abs(literal)] == (literal > 0)){
62                 clause_value = true;
63                 break;
64             }
65         }
66
67         if(!clause_value){
68             return false;
69         }
70     }
71
72     return true;
73 }
74
75
76 vector<Clause> find_unsatisfied_clauses(NormalForm& SAT_instance, Valuation& valuation){
77
78     vector<Clause> unsat_clauses;
79
80     for(const auto& clause : SAT_instance){
81
82         bool clause_value = false;
83         for(auto literal : clause){
84             if(valuation[abs(literal)] == (literal > 0)){
85                 clause_value = true;
86                 break;
87             }
88         }
89
90         if(!clause_value){
91             unsat_clauses.push_back(clause);
92         }
93     }
94
95     return unsat_clauses;
96 }
97

```

```

98
99 Clause choose_random_unsatisfied_clause(vector<Clause>& unsatisfied_clauses){
100
101     int array_size = unsatisfied_clauses.size();
102
103     uniform_int_distribution<> dis(0, array_size - 1);
104     int p = dis(gen);
105
106     return unsatisfied_clauses[p];
107 }
108
109
110 int calculate_breakcount(NormalForm& SAT_instance, Variable var, Valuation& valuation){
111
112     valuation[var] = !valuation[var];
113     int count = 0;
114
115     for(const auto& clause : SAT_instance){
116         bool clause_value = false;
117         for(auto literal : clause){
118             if(valuation[abs(literal)] == (literal > 0)){
119                 clause_value = true;
120                 break;
121             }
122         }
123
124         if(!clause_value){
125             count++;
126         }
127     }
128
129     valuation[var] = !valuation[var];
130
131     return count;
132 }
133
134 bool zero_breakcount_variables(map<Variable, int>& breakcounts){
135
136     for(const auto& el : breakcounts){
137         if(el.second == 0){
138             return true;
139         }
140     }
141
142     return false;
143 }
144
145
146 Variable choose_random_zero_breakcount_variable_to_flip(map<Variable, int>& breakcounts){
147
148     vector<Variable> zeros;
149     for(const auto& el : breakcounts){
150         if(el.second == 0){
151             zeros.push_back(el.first);
152         }
153     }
154
155     int array_size = zeros.size();
156
157     uniform_int_distribution<> dis(0, array_size - 1);
158     int p = dis(gen);
159
160     return zeros[p];
161 }
162
163 Variable choose_random_variable_to_flip(Clause& unsat_clause){
164
165     int array_size = unsat_clause.size();
166
167     uniform_int_distribution<> dis(0, array_size - 1);
168     int p = dis(gen);
169
170     return abs(unsat_clause[p]);
171 }
172
173 #endif

```

Izvorni kod prikazan u radu dostupan je na GitHub repozitorijumu: Implementacija stohastičkih SAT rešavača. Za pokretanje koda, potrebno je na sistemu imati instaliran prevodilac za programski jezik C++. Za sisteme zasnovane na Ubuntu distribuciji Linux sistema, pokrenuti sledeću komandu:

```
sudo apt install build-essential
```

Zatim se kod prevodi komandom:

```
g++ -std=c++20 WalkSAT.cpp
```

i izvršava komandom:

./a.out

U radu je primer dat sa parametrima takvim da se kroz nekoliko pokretanja programa može videti da algoritam daje različite odgovore za isti broj iteracija i istu vrednost verovatnoće. Razlog je upravo nasumičnost početne valuacije i izbora promenljivih.

5 Zaključak

U ovom radu predstavljene su teorijske osnove iskazne logike i SAT problema, kao i osnovni pristup rešavanja problema zadovoljivosti. Poseban fokus stavljen je na stohastičke SAT rešavače kao jednu od metoda pri rešavanju određenih instanci SAT problema. Nasumičnost koja je prisutna u stohastičkim rešavačima omogućava nam da brže pretražimo različite delove prostora pretrage, kao i da se brzo "izvučemo" iz lokalnih minimuma. Takođe je prikazana implementacija WalkSAT algoritma, uz opis korišćenih struktura podataka i ključnih koraka algoritma. Na odabranoj instanci problema pokazana je stohastička priroda algoritma i razlike u rezultatima usled nasumičnosti u izborima. Efikasnost ovih rešavača zavisi i od struktura podataka koje se koriste, kao i načina izbora i *flip*-ovanja promenljivih, što predstavlja prostor za dalje unapređenje stohastičkih metoda.[3]

Literatura

- [1] A. Biere, M. Heule, H. Van Maaren, T. Walsh, *Handbook of Satisfiability*, Amsterdam, The Netherlands, 2021.
- [2] H. H. Hoos, T. Stützle, *Local search algorithms for SAT: An empirical evaluation*, *Journal of Automated Reasoning*, vol. 24, pp. 421-481, 2000.
- [3] A. Fukunaga, *Efficient Implementations of SAT Local Search in Proceedings of the 7th International Conference on Theory and Applications of Satisfiability Testing (SAT'04)*, Vancouver, Canada, 2004.
- [4] P. Janičić, *Matematička logika u računarstvu*, Beograd: Matematički fakultet, 2008.